## TASK 1 – Understand the Problem and Identify the Data

### 1. Inputs and Outputs

The electricity billing program calculates a household's electricity bill using the previous meter reading, current meter reading, household number, and consumer category.

#### Inputs

| Data item | Data type | Justification |
|---|---|---|
| Previous meter reading | Integer | Stores the meter reading recorded during the previous billing period. It must be non-negative. |
| Current meter reading | Integer | Stores the latest meter reading. It must be greater than or equal to the previous reading. |
| Household number | String or Integer | Identifies the household. A string is preferable if the number contains leading zeros or letters. |
| Consumer category | String or Enum | Identifies the type of consumer, such as domestic or commercial, and determines the applicable billing rates. |

#### Outputs

| Data item | Data type | Justification |
|---|---|---|
| Units consumed | Integer | Calculated as current reading minus previous reading. |
| Energy charge | Real number | Represents the cost of electricity consumed according to the tariff rates. |
| Fixed charge | Real number | Represents the fixed fee applicable to the consumer category. |
| Total bill | Real number | The sum of energy charge and fixed charge, possibly including other applicable charges. |

### 2. Constants

Constants are values that do not change during a program run.

- Tariff rates: The price per unit of electricity for each consumption slab.
- Fixed charges: The fixed fee applicable to each consumer category.
- Slab limits: The unit thresholds used to determine the applicable rates.

- Tax or surcharge rates: Any fixed percentage applied according to the billing rules.

These values should be constants because they define the billing rules. If tariffs change, the program's configuration can be updated accordingly.

### 3. Values That Can Be Negative, Fractional, or Very Large

- Negative values: Meter readings, units consumed, energy charges, fixed charges, and the total bill must not be negative. A negative difference between readings indicates invalid input or a meter rollover that requires special handling.
- Fractional values: Energy charges and the total bill may involve decimal values because tariffs, taxes, and surcharges can produce fractions. Monetary calculations should use decimal or fixed-point arithmetic to avoid floating-point rounding errors.
- Very large values: Meter readings, household numbers, and total bills may be large. Suitable data types and range checks should prevent overflow. Household identifiers should be stored as strings when their numeric magnitude has no mathematical meaning.

### 4. Rules for a Correct Bill

A correct billing program must satisfy these conditions:

1. Both meter readings must be valid, non-negative integers.
2. The current reading must be greater than or equal to the previous reading, unless a documented meter rollover or reset is handled.
3. Units consumed must equal current reading minus previous reading.
4. The applicable tariff must match the consumer category and the units consumed.
5. The energy charge must be calculated according to the correct slab rates.
6. Fixed charges and any additional charges must follow the defined billing rules.
7. The total bill must equal the sum of all applicable charges, after the specified rounding rules are applied.
8. The total bill must never be negative.
9. Invalid inputs must be rejected rather than silently producing an incorrect bill.
10. The household number must identify the correct consumer.

For example, if the previous reading is 1,200 and the current reading is 1,530:

If the energy charge is ₹1,650 and the fixed charge is ₹100, then:

This example assumes the tariff calculation has already produced an energy charge of ₹1,650.

### 5. Assumptions

- Meter readings are cumulative readings from the same electricity meter.
- Readings are provided as whole numbers in kilowatt-hours (kWh).
- Tariff rates and fixed charges are provided by the electricity provider.
- The consumer category is selected from a predefined list of valid categories.
- The bill is calculated for one household and one billing period.
- The billing system specifies how taxes, surcharges, and monetary rounding are handled.
- Meter rollover, faulty readings, and meter replacement are handled separately if they occur.

### 6. Alternative Concept and Complexity Analysis: Readings Stored as Text

Suppose meter readings are stored as character strings, such as "01530", instead of integers.

#### Time complexity

Let be the number of digits in a meter reading.

- Integer representation: Reading the values is straightforward, and subtraction typically takes constant time for ordinary machine-sized integers. For arbitrary-precision integers, subtraction takes time.
- String representation: The program must first validate the characters and convert each string into an integer. Conversion takes time per reading. Comparing two strings numerically also requires conversion or a carefully designed digit-based comparison.
- Overall: For two readings, conversion and calculation take time. If the readings contain a fixed maximum number of digits, this is effectively constant time.

#### Space complexity

- Integer representation: Each reading uses a fixed amount of memory for a fixed-width integer.

- String representation: Each reading requires space proportional to the number of digits, or , plus any memory needed for conversion.
- Additional overhead: String objects may require extra memory for character storage, length information, and object metadata.

### Conclusion

Integer storage is preferable for meter readings because it supports direct arithmetic, simplifies validation, and avoids repeated conversions. Text storage is useful when preserving leading zeros or formatting is important, but the strings should be converted to validated numeric values before performing billing calculations.